

Decoupling Logical Masks from GPU Execution for Dynamic Block-Sparse Attention

Shanghao Liu Xiaoyun Yu Wanting Li Wenqi Jiang^{*}
 National University of Singapore
 Singapore, Singapore
 wenqi.jiang@nus.edu.sg^{*}

Abstract

Attention computation makes inference expensive in *video diffusion transformers* (vDiTs), which generate videos through iterative denoising. *Block-sparse attention* (BSA) reduces this cost by computing only blocks selected by a *logical mask*, which specifies the attention interactions to compute. However, coupling logical block geometry to execution choices limits adaptation to varying masks and graphics processing units (GPUs), while runtime kernel specialization can incur preparation overhead that outweighs execution time savings. We present TESSERA, a specialized runtime for dynamic BSA that decouples logical masks from GPU execution while preserving the specified attention interactions. First, its physical mapping layer retains, combines, or subdivides logical attention blocks into physical tiles suited to different attention mask shapes and GPU architectures. Second, its task organization layer groups and schedules tiles within GPU tasks to reuse data, expose parallelism, and overlap data movement with computation. Third, its profile-guided regime selection enables low-overhead execution plan selection through a lookup table constructed from offline profiling. We implement these three components in TESSERA with specialized CUDA kernels supporting four NVIDIA GPU generations. Evaluated on 2,315 real attention masks and industrial video diffusion models, TESSERA achieves up to $6.79\times$ BSA request speedup over baseline systems in the evaluated video diffusion models.

1 Introduction

Video generation powered by video diffusion transformers (vDiTs) has emerged as a popular yet computationally demanding generative AI application [22, 54]. For example, generating a five-second 720p clip with HunyuanVideo on a single H100 GPU takes approximately 16 minutes [90], equivalent to 3.15 GPU-minutes per second of generated video.

Attention is a major performance bottleneck in this inference workload, accounting for roughly 85% of its runtime [90]. To generate a clip, a vDiT repeatedly denoises a spatiotemporal latent representation; at each denoising step, its attention layers process the long token sequence representing the full clip—115K tokens for a five-second clip [90]. Each attention layer computes scores from the query matrix Q and key matrix K through QK^T , then uses the resulting

attention weights to aggregate the value matrix V ; its computational cost therefore grows quadratically with sequence length [76].

Block-sparse attention (BSA) is a widely adopted approach to reduce the high attention cost in video models. Examples include academic methods such as VSA, SpargeAttn, SVG, SVG2, and SPADE [40, 76, 82, 88, 89], as well as industrial video models such as HunyuanVideo-1.5 and LongCat-Video [47, 73]. BSA divides the attention matrix into rectangular *logical blocks*, each covering B_Q query tokens and B_{KV} key/value tokens, with block geometry $B = (B_Q, B_{KV})$. A *logical mask* M marks each logical block as active or inactive, and BSA computes only the active blocks.

The highly dynamic masks used in BSA make efficient GPU execution challenging. Active blocks can change across video-generation requests, attention heads, layers, and denoising steps, while logical block sizes are parameters chosen by the sparsity algorithm [40, 47, 73, 82, 89]. Each BSA invocation, termed a *runtime request*, therefore requires an execution plan suited to its mask structure, block geometry, input tensors, and target GPU. A BSA runtime thus faces two requirements: (*R1*) provide high-performance execution plans for these diverse scenarios, and (*R2*) keep plan selection and preparation overhead low.

Existing BSA systems deliver suboptimal performance on dynamic masks because they do not meet the aforementioned requirements. One category uses specialized kernels, such as Block Sparse Attention and flex-block-attn [23, 53], which provide ready-to-use, hand-tuned implementations for pre-set block sizes. These implementations avoid per-request compilation, but their fixed execution strategies across input masks limit performance across diverse masks and GPUs. Another category, exemplified by FlexAttention and Flash-Infer [15, 83], uses compilation or runtime planning to specialize execution and improve kernel performance. However, compilation, plan selection, and metadata preparation can incur substantial overhead that offsets the execution-time savings. In addition, configurable physical tile sizes can remain constrained by logical block geometry, as in FlexAttention’s subdivision of logical blocks, limiting the available mappings from logical blocks to GPU execution (Section 2.2).

We present TESSERA, the first specialized runtime system for kernel-level physical planning of dynamic block-sparse attention in video generation. TESSERA

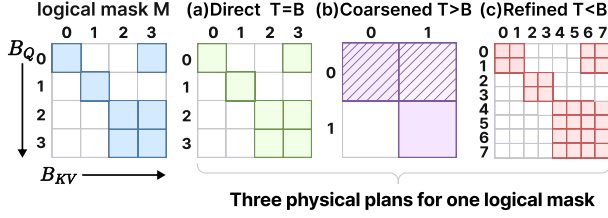


Figure 1. The same logical sparse-attention mask (colored blocks in the left sub-figure) can be executed using physical tiles that are (a) equal to, (b) larger than, or (c) smaller than the logical blocks. Here, B denotes the logical block size, and T denotes the physical tile size used for GPU execution.

provides a semantics-preserving execution abstraction that separates the algorithm’s logical attention specification from its GPU implementation through three complementary components. First, the *physical mapping* decouples logical blocks from physical tiles, allowing the same mask to be executed using tiles that retain, combine, or subdivide logical blocks while preserving the specified attention interactions. Second, the *task organization* determines how these tiles are grouped into CUDA tasks, each executed by one GPU thread block, how data movement overlaps with computation, and how partial results are reduced. Together, these two components define efficient *physical plans* for executing the same mask, addressing *R1*. Third, *profile-guided regime selection* uses offline profiling to rank these plans by *performance regime*, a group of requests that shares a profiled plan ranking. At runtime, TESSERA selects the highest-ranked eligible plan from its precompiled kernel catalog through a low-overhead lookup and eligibility check, addressing *R2*. To enable efficient BSA execution across GPU architectures, we develop architecture-specific catalogs of specialized CUDA kernels for TESSERA, supporting four generations of NVIDIA GPUs.

First, flexible mapping from logical blocks to physical tiles improves performance by adapting tile geometry to the mask and the GPU architecture. Let $T = (T_Q, T_{KV})$ denote the physical tile geometry, where each tile covers the interactions between T_Q query tokens and T_{KV} key/value tokens processed at a time by the GPU executor. The mapping layer offers three alternatives: *Direct* keeps $T = B$ (Figure 1(a)), *Coarsened* covers several logical regions with a larger physical tile while masking out inactive entries (Figure 1(b)), and *Refined* divides logical blocks into smaller physical tiles (Figure 1(c)). These alternatives trade data movement against computation: coarsening can improve operand reuse and amortize indexing overhead but introduces redundant computation on masked-out entries, whereas refinement reduces the per-tile register and shared-memory footprint but can increase data movement and tile-processing overhead. Consequently, the best mapping depends on both mask structure and GPU architecture

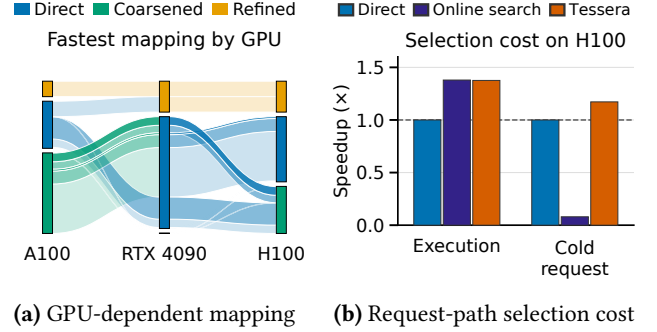


Figure 2. The fastest physical mapping for 2,315 aligned masks depends strongly on GPU architecture, as shown in (a). On the same H100 mask population, exhaustive online plan search incurs substantial overhead that TESSERA avoids through profile-guided selection (b).

(Figure 2a). In our H100 experiments, the physical plans selected by TESSERA achieve approximately $1.38\times$ geometric-mean kernel speedup over the fixed Direct baseline.

Second, the task organization layer improves performance by determining how physical tiles are grouped into CUDA tasks, the order to process them, and how data movement overlaps with computation. For example, grouping several tiles into one task enables reuse of the query and output accumulator, but leaves fewer tasks to distribute across streaming multiprocessors (SMs). Conversely, partitioning the active K/V tiles for a query tile across multiple tasks exposes more parallelism across SMs, but incurs overhead from merging partial results. Pipeline depth provides another trade-off: deeper pipelines increase data-movement–compute overlap but require additional buffering resources. Given the same physical mapping, optimizing task organization yields a $1.105\times$ geometric-mean speedup on H100 in our experiments.

Third, profile-guided regime selection chooses a high-performance execution plan with low overhead. The mapping and organization layers expose many execution plans, but exhaustively preparing and benchmarking eligible plans for each new mask is expensive: in our H100 experiment, this online search incurs a cold-request latency $12.52\times$ that of the Direct baseline (Figure 2b). TESSERA moves this search offline by profiling the target GPU’s supported plans on a representative mask corpus and constructing plan rankings for different performance regimes. At runtime, the request’s block geometry, tensor shape, and optional mask summaries identify the regime. TESSERA then selects the highest-ranked eligible plan without online benchmarking. This approach serves cold requests $14.67\times$ faster than exhaustive online search while achieving similar kernel performance, as shown in Figure 2b.

We evaluate TESSERA across four NVIDIA GPU generations and 2,315 real attention masks. TESSERA improves

performance over baseline systems in all 76 evaluated combinations of GPU platforms, systems, and workload families. Across these attention masks, TESSERA achieves geometric-mean speedups of $1.85\text{--}5.11\times$ over FlashInfer, $1.62\text{--}33.73\times$ over FlexAttention, and $3.96\times$ over the *flex-block-attn* library across the four workload families that *flex-block-attn* supports on H100. TESSERA also achieves kernel-only geometric-mean speedups of $1.12\text{--}6.38\times$ over the baselines across all four platforms. We further evaluate TESSERA within 720p Wan2.1 and HunyuanVideo-1.5 inference on H100, where it accelerates attention by $2.95\text{--}6.79\times$ and the full 50-step diffusion loop by $1.22\text{--}2.08\times$.

This paper makes three contributions:

- **Two-layer physical plans and native kernel catalogs.** We propose two-layer physical plans combining physical mapping and task organization, and implement them as architecture-specific catalogs of precompiled native CUDA kernels across four GPU platforms.
- **Profile-guided regime selection.** We construct per-GPU plan rankings from offline profiles and select the highest-ranked eligible plan at runtime through a low-overhead lookup, without expensive per-mask compilation or exhaustive plan search.
- **The TESSERA runtime for dynamic BSA.** We integrate these solutions into TESSERA, a specialized runtime system for dynamic BSA. Across four GPU platforms, TESSERA reduces complete-request latency in all 76 evaluated platform–baseline–family combinations and accelerates attention by $2.95\text{--}6.79\times$ in the evaluated video diffusion models.

2 Background and Motivation

2.1 Dynamic Masks and Logical Block Geometry

The logical mask specifies the attention interactions that each execution must preserve. For one attention head, let $Q \in \mathbb{R}^{N_q \times d}$ be the query matrix and $K, V \in \mathbb{R}^{N_{kv} \times d}$ the key and value matrices, where N_q and N_{kv} are sequence lengths and d is the head dimension. A block-sparse mask M marks a set of logical block pairs active, each covering B_Q query tokens and B_{KV} key/value tokens under the logical block geometry $B = (B_Q, B_{KV})$. Expanding the active pairs to token coordinates gives the *effective interaction set* $E(M)$, and masked attention computes

$$O_M = \text{softmax}\left(\frac{QK^T}{\sqrt{d}} + \mathcal{B}_M\right)V, \quad (1)$$

where $\mathcal{B}_M(i, j) = 0$ for $(i, j) \in E(M)$ and $-\infty$ otherwise. $E(M)$ specifies the required interactions independently of how the work is laid out on the GPU.

The *sparsifier*, which selects the active attention blocks, supplies both M and B to the runtime. Video sparsifiers recompute the active block pairs across runtime requests,

heads, layers, and denoising steps, so the mask can change between runtime requests [40, 47, 73, 76, 82, 88, 89]. B fixes the granularity at which the sparsifier expresses M ; some sparsifiers fix it, others select it per target model [40, 47, 73, 82, 89].

2.2 The Cost of Logical-to-Physical Coupling

Logical blocks and physical tiles are choices made at different levels: the logical block is an algorithmic choice of the sparsifier that states where attention is computed, whereas the physical tile is an execution choice of the kernel that states how that work is laid out on the GPU. A backend can nevertheless couple the two through its kernel interface. The logical block geometry B defines the regions named by the mask; the *physical tile geometry* $T = (T_Q, T_{KV})$ defines the query–key/value region processed at a time; and the native instruction geometry defines the shape consumed by a Tensor Core instruction. A *GPU task* groups one or more physical tiles for scheduling.

Existing execution paths offer tile configuration while retaining constraints from logical block geometry. Block Sparse Attention and *flex-block-attn* provide kernels for preset block sizes [23, 53]. FlexAttention’s standard Triton forward path supports configurable tile sizes, but requires B_Q and B_{KV} to be divisible by T_Q and T_{KV} , respectively: tiles can match or subdivide logical blocks [15]. These tile sizes are kernel specialization parameters chosen explicitly or through autotuning. FlashInfer also supports multiple tile configurations and runtime planning; its described block-sparse design aligns block rows with query tiles [83].

The available physical mappings affect tile footprint, task granularity, and operand reuse. Direct mapping, $T = B$, simplifies sparse execution because an active block index identifies a physical tile without metadata describing membership inside that tile. However, the sparsifier selects B for mask expressiveness rather than for the GPU, so the same $E(M)$ often executes slower under $T = B$ than under an alternative physical covering (Section 2.4). TESSERA exposes mappings that retain, combine, or subdivide logical blocks while preserving $E(M)$ (Figure 1), and selects among eligible precompiled plans for each runtime request using offline plan rankings.

2.3 Analysis of Data–Compute Pipeline Performance

Block-sparse attention can execute as a two-stage *data–compute pipeline* whose data movement and attention computation overlap (Figure 3). This view follows FlashAttention-3 [55] and CUTLASS [49]. The *data-movement stage* transfers operands and mask metadata between global memory and on-chip storage. The *attention-compute stage* computes QK^T , incrementally updates softmax across tiles, and multiplies the resulting attention weights by V . When movement for the next tile overlaps compute for the current tile, the longer stage determines steady-state throughput. For architecture

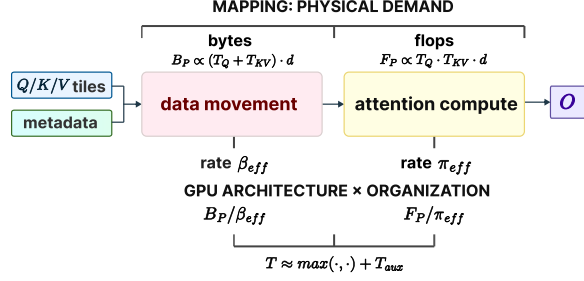


Figure 3. Block-sparse attention as a two-stage data-compute pipeline. The plan and GPU jointly determine byte demand $B_P(M)$, arithmetic demand $F_P(M)$, their effective service rates, and auxiliary cost.

A, mask M , and physical plan P ,

$$T_A(P, M) \approx \max\left(\frac{B_P(M)}{\beta_A^{\text{eff}}(P, M)}, \frac{F_P(M)}{\pi_A^{\text{eff}}(P, M)}\right) + T_{\text{aux}}(P, M). \quad (2)$$

Equation 2 separates stage demand from the rate at which the GPU serves it. B_P and F_P count the bytes and arithmetic of the main attention kernel. B_P includes Q/K/V transfers, the final-output store, metadata, and plan-specific state; F_P includes work on inactive entries covered by physical tiles. The effective bandwidth β_A^{eff} and compute rate π_A^{eff} depend on both P and A [71]. T_{aux} covers additional overheads outside the overlapped steady-state terms, including scheduling, pipeline fill and drain, load imbalance, synchronization, and any separate reduction and its traffic, counted only in T_{aux} .

On-chip reuse amortizes traffic that occurs once per query-tile traversal. A FlashAttention-style traversal keeps one Q tile, its online-softmax state, and its output accumulator on chip while successive K/V tiles pass through the mainloop. Under simplified bfloat16 (BF16) accounting that includes K/V loads but excludes metadata and cache effects, each physical tile performs about $4dT_Q T_{KV}$ floating-point operations across QK^T and the multiplication of attention weights by V , while loading about $4dT_{KV}$ K/V bytes. The Q load and final-output store occur once per traversal. Under this simplified accounting, their per-tile contribution diminishes as more K/V tiles are visited, and the ratio of matrix arithmetic to traffic approaches T_Q floating-point operations per byte.

Physical mapping and task organization jointly affect demand and the balance between the data-movement and attention-compute stages. Larger tiles can reduce tile and indexing counts but add arithmetic when they cover inactive entries. Smaller tiles can reduce such inactive work and per-task footprint but increase tile count and repeated operand traffic. Task grouping, traversal order, parallel decomposition, and pipeline depth further change locality, reuse, parallelism, overlap, and auxiliary cost. Their combined effects determine the physical cost of executing the same $E(M)$.

2.4 Challenges in Physical Plan Selection

Our experiments show that plan preferences vary with the mask, GPU, and task organization, and identifying the best choice introduces a selection cost of its own. First, mask structure changes the fastest plan at fixed GPU and geometry: within the H100 $B = (32, 16)$ family, 220 masks favor Coarsened and 30 favor Direct according to our experiments. Second, GPU architecture changes the preferred mapping even for identical inputs: all 250 masks in this family favor Coarsened on A100 but Direct on RTX 4090 (Figure 2a). Third, optimizing task organization at fixed mapping yields a 1.105× geometric-mean kernel execution speedup on H100.

Exhaustive online search for each runtime request incurs high selection cost. In our experiments across 2,315 H100 masks, probing every candidate yields a nine-family geometric-mean kernel execution speedup of 1.379× over Direct. For a cold request, however, complete-request latency rises to 12.52× that of the Direct baseline because it includes plan selection, launch preparation, candidate probes, and native execution (Figure 2b). Section 7 reports these measurements in detail. The resulting challenge is to select among viable executions while keeping the overhead of plan selection and launch preparation below the kernel execution time saved over Direct as the mask and target GPU change.

3 TESSERA Overview

TESSERA separates the construction of physical plans for dynamic block-sparse attention from their selection at runtime. As shown in Figure 4, the two-layer physical plans vary the mapping and organization of GPU work, while profile-guided selection evaluates these plans offline and reuses the resulting rankings online. This separation allows TESSERA to adapt execution to each mask and GPU without probing candidate plans on the request path.

Physical-plan portfolio. The *physical-plan portfolio* defines the available execution choices (Figure 4, top). Each choice is a precompiled CUDA implementation that combines a mapping layer with an organization layer. The mapping layer determines which physical tiles cover the active logical blocks. The organization layer determines how those tiles form GPU tasks and how the tasks are grouped, traversed, and executed in parallel.

Offline deployment. Offline deployment converts the portfolio into architecture-specific plans and rankings (Figure 4, middle). For a target architecture A , TESSERA first filters the portfolio for plans supporting A , producing the *native CUDA catalog* C_A . It then validates and profiles these plans using dynamic masks and request parameters from video models. From these measurements, TESSERA groups requests with similar plan performance into performance regimes and stores each regime’s ranking in the *plan table* Π_A . The catalog and table are versioned and deployed together.

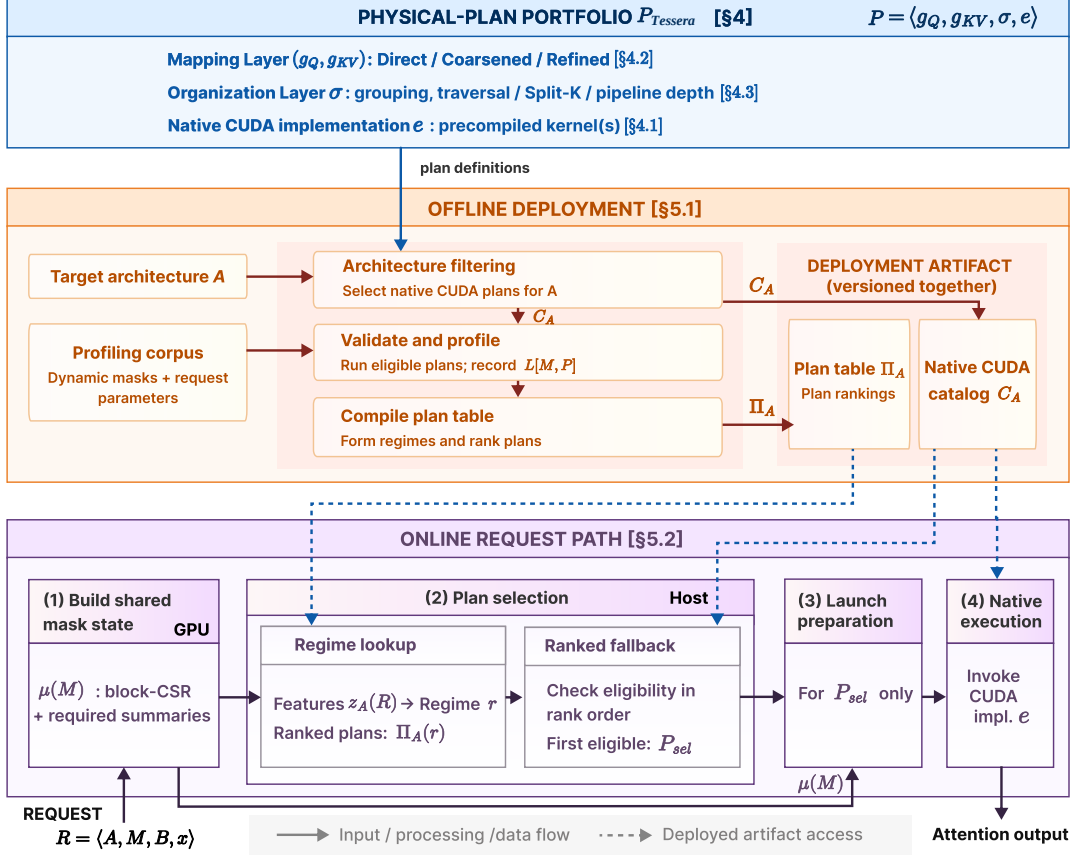


Figure 4. TESSERA overview. Offline deployment filters the physical-plan portfolio into the native CUDA catalog C_A of the target architecture A , then validates and profiles the catalog plans on a profiling corpus and compiles the plan table Π_A . Online, the runtime builds the shared mask state $\mu(M)$ on the GPU, selects the first eligible plan from the ranked plans of its performance regime, prepares only that plan, and invokes its native CUDA implementation.

Online request path. The online request path uses the deployed catalog and plan table to select and execute one plan for the current request (Figure 4, bottom). A request R provides the logical mask M , the logical block geometry B , and tensor properties such as the data type, head dimension, and layout. The runtime first builds the *shared mask state* $\mu(M)$ on the GPU, which contains the active-block indices and any scalar mask statistics required by the plan table. It combines these statistics with the architecture, block geometry, and request dimensions such as batch size, head count, and sequence length to identify a performance regime.

The runtime scans the regime’s ranking and selects the first plan P_{sel} that preserves $E(M)$ and supports the request. It then derives the metadata required by P_{sel} from $\mu(M)$, binds any temporary GPU buffer required by the plan, and invokes its implementation in C_A .

4 Two-Layer Physical Plans

4.1 Physical Plan

A physical plan binds a physical mapping and a task organization to the native CUDA implementation that realizes them. We represent this binding as $P = \langle g_Q, g_{KV}, \sigma, e \rangle$, where g_Q and g_{KV} map the logical block geometry $B = (B_Q, B_{KV})$ to the physical tile geometry $T = (T_Q, T_{KV})$ on the query and key/value dimensions, respectively. The two dimensions can use different mapping ratios. The task organization σ specifies grouping, traversal order, the number of parallel K/V reduction partitions, and pipeline depth. The native CUDA implementation e is one precompiled kernel or a fixed sequence of kernels that realizes these decisions on one GPU architecture. A common plan interface exposes these decisions across GPU architectures.

Task organization assigns one or more physical tiles to each *cooperative thread array* (CTA), or CUDA thread block, and determines their traversal order. Table 1 summarizes the decisions in the mapping and organization layers, their execution outputs, and their main tradeoffs.

Table 1. Physical-plan decisions and performance tradeoffs.

Decision	Output	Tradeoff
<i>Physical mapping layer (§4.2)</i>		
Direct	One tile per active logical block	Simple indexing vs. more tiles for small blocks
Coarsened	Larger tiles with membership metadata	Operand reuse vs. compute on inactive entries
Refined	Smaller tiles along refined dimensions	Resource savings vs. more tiles and repeated traffic
<i>Task organization layer (§4.3)</i>		
Grouping and traversal	Grouped tasks and tile traversal order	Locality and metadata reuse vs. coarse scheduling and imbalance
Split-K	Parallel K/V-reduction tasks and a merge	Parallelism vs. partial-state traffic, merging, and synchronization
Pipeline depth	Buffered tile prefetch stages	Load–compute overlap vs. register and shared-memory use

4.2 Physical Mapping Layer

The mapping layer offers three ways to relate physical tile dimensions to logical block dimensions: Direct, Coarsened, and Refined.

Direct. Direct uses $T = B$, mapping each active logical block to one physical tile (Figure 1(a)). The active-block index directly identifies the tile, and all computation within the tile corresponds to active attention entries. Smaller logical blocks therefore produce more, smaller tiles.

Coarsened. Coarsened covers multiple logical regions with one larger physical tile (Figure 1(b)), which reduces the tile count and reuses Q or K/V operands across adjacent regions. For a tile containing inactive entries, *membership metadata* records the active logical blocks so that the kernel can mask inactive scores before the softmax update. Normalization and the weighted sum of values therefore include only the effective entries in $E(M)$.

Coarsened saves operand traffic through reuse while performing additional matrix computation on the inactive entries inside each tile. Lower active fractions increase this extra work per effective entry. Whether the saved traffic outweighs the extra work depends on the active fraction and the target GPU’s effective compute and transfer rates.

Refined. Refined covers a logical region with smaller physical tiles in at least one dimension (Figure 1(c)). Smaller tiles use fewer registers and less shared memory in the attention-compute stage and can match the native instruction geometry. The costs are more tiles and, potentially, repeated loads and state traffic. Under a FlashAttention-style traversal, query-side refinement creates independent query

tiles, whereas K/V-side refinement updates the same query tile’s *online-softmax state* sequentially across K/V tiles. This state holds the running maximum score, the scaled exponential sum, and the output accumulator.

For example, one H100 plan maps each 128×128 logical block to four 64×64 physical tiles. The *block compressed sparse row (block-CSR)* indices store the active blocks of each query-block row. Launch preparation expands each block-CSR column index, which names a 128-wide active block, into the indices of two 64-wide K/V tiles. The kernel launch assigns each CTA one 64-high query tile, so the two query halves of a logical block run in separate CTAs, and each CTA processes its query tile while traversing the expanded K/V tiles. Together, the four tiles cover the original logical block and preserve $E(M)$.

4.3 Task Organization Layer

The organization layer groups and schedules physical tiles for GPU execution according to σ . It chooses how to group and traverse tiles, how many CTAs share a K/V reduction, and how many stages buffer prefetched tiles. These decisions control locality, parallelism, and data-movement–compute overlap.

Grouping and traversal. Grouping and traversal organize active blocks into tasks by contiguous runs. An *active-block run* is a maximal sequence of consecutive active K/V blocks in one query-block row. For example, column indices $\{2, 3, 4, 9\}$ form two runs, $[2, 4]$ and $[9, 9]$. Grouping combines one or more runs, possibly from several query-block rows, into one task, while traversal determines the order in which the CTA visits them. The CTA retains the query tile and its online-softmax state on chip, reusing them as it traverses active blocks within and between runs. Combining more work in one task amortizes metadata accesses and increases on-chip reuse, but leaves fewer tasks for the scheduler to assign independently. Tasks with more active blocks or longer runs finish later, causing load imbalance.

Split-K increases parallelism by partitioning one query tile’s K/V reduction across multiple CTAs (Figure 5). Each CTA writes its maximum score m , scaled exponential sum l , and partial output accumulator O to the *workspace*, a temporary GPU buffer. A subsequent kernel rescales these partial states to a common maximum, merges them, and normalizes the output. The additional parallelism incurs partial-state traffic, merge computation, and synchronization.

Pipeline depth. *Pipeline depth* is the number of buffered stages used to prefetch upcoming tiles. A deeper pipeline prefetches more of the subsequent K/V tiles and increases data-movement–compute overlap, at the cost of additional register and shared-memory usage.

4.4 Native CUDA Catalogs

For a target architecture A , TESSERA forms the native CUDA catalog C_A by retaining plans that have an implementation

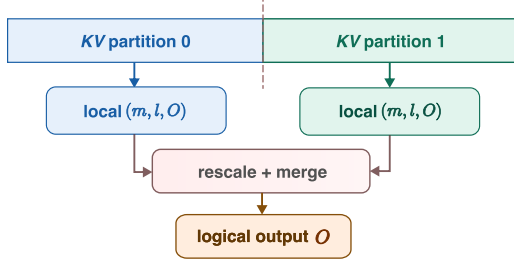


Figure 5. Split-K organization. CTAs process partitions of one query tile’s K/V reduction domain and write local (m, l, O) states to the workspace. A reduction kernel rescales and merges these states to produce the output.

for A and meet its hardware capability requirements. Counting each plan–logical-geometry pair as one entry, the catalogs contain 14–26 entries across 3–9 physical tile shapes.

Offline table compilation and online plan selection further filter C_A to identify plans that support each request (Algorithms 1 and 2). For a request $R = \langle A, M, B, x \rangle$, x specifies the data type, head dimension, and tensor layout. A plan $P \in C_A$ satisfies $\text{Eligible}(P, R)$ when its implementation supports the request’s logical geometry, tensor properties, metadata format, and available workspace while preserving the output specified by M .

5 Profile-Guided Regime Selection

Profile-guided regime selection moves plan profiling offline and uses the resulting rankings to select plans at runtime. TESSERA profiles supported plans and groups requests with similar plan performance into performance regimes, each with a plan ranking. At runtime, the selector uses request features such as block geometry and tensor shape to look up the regime in the plan table and selects the highest-ranked eligible plan. Algorithms 1 and 2 detail offline table construction and online selection, respectively.

5.1 Offline Plan-Table Compilation

Offline profiling records validated latencies for table compilation (Algorithm 1, lines 2–9). A *profiling case* pairs a mask M with its request parameters. For each eligible plan P in C_A , TESSERA compares the output against a masked-attention reference and records the median kernel latency as $L[M, P]$, where M indexes the mask and its request parameters.

Table compilation uses a two-stage hierarchy to capture the request properties that change plan performance (Algorithm 1, lines 10–14). The first stage uses the request key $k_A(R)$ to form coarse groups with the same architecture, logical geometry, and tensor properties. The second stage partitions each group more finely using bucketed request dimensions $u_A(R)$, such as batch size, head count, and sequence length, and optionally scalar mask statistics $s_A(M)$. These statistics include *active-block density*, the fraction of

Algorithm 1 Table compilation for target architecture A .

Require: native CUDA catalog C_A , profiling corpus $\mathcal{M}$, feature schema F_A

Ensure: plan table Π_A

1: $L \leftarrow \emptyset; \Pi_A \leftarrow \emptyset$

Profiling

2: **for all** cases $R = (A, M, B, x)$ in $\mathcal{M}$ **do**

3: **for all** $P \in C_A$ with $\text{Eligible}(P, R)$ **do**

4: Run P and compare with the reference output

5: **if** the output passes numerical validation **then**

6: $L[M, P] \leftarrow$ median kernel latency

7: **end if**

8: **end for**

9: **end for**

Regimes and rankings

10: **for all** request keys k in $\mathcal{M}$ **do**

11: $\mathcal{M}_k \leftarrow$ cases with $k_A(R) = k$

12: $P_{\text{base}} \leftarrow$ catalog’s base plan for k

13: $\mathcal{B}_k \leftarrow$ partition $\mathcal{M}_k$ using F_A

14: **for all** nonempty buckets $b \in \mathcal{B}_k$ **do**

15: $C_b \leftarrow$ plans with a valid $L[M, P]$

 for every case M in bucket b

16: **if** $C_b \neq \emptyset$ **then**

17: Sort C_b by increasing geo-mean latency over b

18: Break exact ties by plan identifier

19: $\Pi_A[k, b] \leftarrow C_b$ followed by P_{base}

20: **end if**

21: **end for**

22: **end for**

23: **return** Π_A

logical blocks that are active, and *active-block-run coverage*, the fraction of active blocks that belong to runs of at least two, defined as zero for an empty mask. Each resulting partition forms a performance regime.

The *feature schema* F_A makes this partitioning consistent between table compilation and runtime lookup by fixing the features, their types, and their bucket boundaries. For the second stage, these boundaries are predefined intervals or thresholds learned by a decision tree, whose leaves define the finer partitions. We select the tree depth through cross-validation on corpus folds. Masks from the same source remain in one fold, and each leaf contains at least 32 cases.

Each performance regime maps to a ranking of plans by latency in the plan table Π_A (Algorithm 1, lines 15–19). TESSERA ranks plans with valid measurements on every case in the regime by geometric-mean latency, favoring plans that perform well across similar requests. Each ranking ends with the request key’s *base plan*, a catalog plan eligible on every profiling case with that key.

TESSERA packages the native CUDA catalog, plan table, and feature schema into one versioned deployment artifact. Runtime lookup therefore uses the same catalog and feature definitions as offline compilation.

Algorithm 2 Online plan selection for $R = (A, M, B, x)$.

Require: plan table Π_A , native CUDA catalog C_A , feature schema F_A

Ensure: selected plan P_{sel} or no-eligible-plan

```

1: Build GPU  $\mu(M)$ : block-CSR and  $F_A$  summaries
2:  $s_A(M) \leftarrow \emptyset$ 
3: if  $F_A$  requires summaries then
4:   Copy required scalars from  $\mu(M)$  to host  $s_A(M)$ 
5: end if
6:  $z \leftarrow \langle k_A(R), u_A(R), s_A(M) \rangle$ 
7:  $r \leftarrow \rho_A(z)$ 
8: if lookup finds  $r$  then
9:    $\mathcal{L} \leftarrow \Pi_A(r)$ 
10: else
11:    $\mathcal{L} \leftarrow [\text{base plan for } k_A(R)]$ 
12: end if
13: for all  $P \in \mathcal{L}$  in rank order do
14:   if Eligible( $P, R$ ) then
15:     return  $P$ 
16:   end if
17: end for
18: return no-eligible-plan

```

5.2 Online Plan Selection

Online plan selection looks up a precomputed ranking and returns one eligible plan (Algorithm 2). The runtime then prepares that plan for execution. These steps comprise regime lookup, ranked fallback, and launch preparation. This lookup-and-check procedure achieves performance close to the fastest measured eligible plans and exhaustive online search, with low selection overhead (Section 7.4).

Regime lookup. The runtime transfers only the scalar mask statistics needed for lookup (Algorithm 2, lines 1–5). It builds the shared mask state $\mu(M)$ on the GPU, containing the block-CSR offsets and active K/V column indices together with these statistics $s_A(M)$. It transfers $s_A(M)$ to the host while the full mask coordinates and block-CSR remain on the GPU.

The lookup combines the request key, bucketed request dimensions, and mask statistics (Algorithm 2, lines 6–9) as

$$z_A(R) = \langle k_A(R), u_A(R), s_A(M) \rangle, \quad (3)$$

and sets $s_A(M) = \emptyset$ when the plan table does not use mask statistics. The lookup function ρ_A matches $k_A(R)$ and applies the boundaries in F_A to identify a performance regime r and its plan ranking:

$$r = \rho_A(z_A(R)), \quad \Pi_A(r) = [P_1, P_2, \dots, P_k]. \quad (4)$$

Because plan rankings are indexed by regime, TESSERA can reuse a ranking across different masks assigned to the same regime. In contrast, a mask-specific cache reuses a ranking only when the exact mask recurs.

Ranked fallback. The ranking estimates performance, while eligibility checks establish whether a plan supports the current request and preserves its specified output. The

Table 2. Hardware platforms. BF16/SM is the peak dense BF16 Tensor Core throughput with 32-bit floating-point (FP32) accumulation per SM, in TFLOP/s. BW/SM is the peak global-memory bandwidth per SM, in GB/s. F/B is their ratio in flop/byte; Cap is the rated maximum board power in W.

GPU	Memory	BF16/SM	BW/SM	F/B	Cap
RTX 4090 (SM89)	24GB	1.291	7.88	163.9	450
RTX 5090 (SM120)	32GB	1.232	10.54	116.9	575
A100 PCIe (SM80)	80GB	2.889	17.92	161.2	300
H100 SXM (SM90a)	80GB	7.496	25.38	295.4	700

runtime scans plans in rank order and selects the first eligible plan (Algorithm 2, lines 13–18). A lookup miss uses the base plan as its sole candidate (lines 10–12) and applies the same eligibility check.

Launch preparation. The runtime derives only the metadata required by the selected plan from the shared block-CSR. Depending on the plan, this metadata contains traversal descriptors, tile membership information, refinement indices, or Split-K partition information, together with the required workspace bindings. The runtime then invokes the plan’s native CUDA implementation in C_A .

6 Implementation

TESSERA consists of a Python runtime for *PyTorch* and a C++ and CUDA extension. Excluding third-party dependencies, the implementation comprises 18K lines of Python and 19K lines of C, C++, and CUDA code. The host performs regime lookup and eligibility checks, while GPU kernels build the shared mask state and the metadata required by the selected plan, which the runtime then invokes. Its native CUDA catalogs cover A100, RTX 4090, H100, and RTX 5090.

7 Evaluation

We evaluate whether TESSERA provides high-performance execution plans across diverse masks, block geometries, and GPUs (R1) while keeping plan selection and preparation overhead low (R2). Request, kernel, and model speedups assess the runtime (contribution 3; Section 7.2); variation across geometries, masks, GPUs, and organizations assesses two-layer plans (contribution 1; Section 7.3); and selection-quality and request-cost ablations assess regime selection (contribution 2; Section 7.4).

7.1 Experimental Setup

Platforms and systems. TESSERA runs on four NVIDIA GPU platforms (Table 2). We compare it with FlashInfer [83], FlexAttention [15], and, on the four H100 families it supports, flex-block-attn [53]. Each platform uses its fixed native CUDA catalog C_A and per-GPU plan table Π_A (Section 5).

Table 3. Workload configurations. Q-resident intensity is the Direct tile’s matrix arithmetic per byte of BF16 K/V loads, T_Q flop/byte, with on-chip Q reuse (Section 2.3).

Configuration	B_Q	B_{KV}	Mean density	Intensity	Cases
$Q_{128}K_{128}$	128	128	9.91%	128.0	250
$Q_{128}K_{64}$	128	64	9.91%	128.0	250
$Q_{64}K_{64}$ -HD	64	64	5.35%	64.0	250
$Q_{64}K_{64}$	64	64	11.95%	64.0	315
$Q_{64}K_{64}$ -DD	64	64	18.20%	64.0	250
$Q_{64}K_{32}$	64	32	10.07%	64.0	250
$Q_{32}K_{32}$	32	32	9.76%	32.0	250
$Q_{32}K_{16}$	32	16	9.89%	32.0	250
$Q_{16}K_{16}$	16	16	9.79%	16.0	250
Total					2,315

Workloads. The microbenchmark contains 2,315 block-sparse video-attention masks from four sources: Wan2.1-T2V-14B, Wan2.2-T2V-14B [69], and HunyuanVideo-1 [29] masks generated by SPADE [40], and HunyuanVideo-1.5 masks from its official sparse-distilled image-to-video model [73]. The masks form nine families spanning seven logical geometries; three use a larger query block than key/value block, reflecting the finer key/value granularity used in sparse video-generation systems [37, 82]. Mask *density* is the fraction of active logical blocks, equal to one minus mask sparsity (the fraction of inactive blocks). Table 3 gives block sizes, densities, and case counts. Family names Q_qK_k specify $B_Q = q$ and $B_{KV} = k$; Q_{16} through Q_{128} abbreviate families by query block size. The suffixes HD and DD identify the lower- and higher-density variants of $Q_{64}K_{64}$. Mean densities otherwise follow the 9–12% default range of the four vDiT sources. Measurements are paired across platforms by case identifier.

We evaluate two video generation models on H100: 720p Wan2.1 text-to-video with SPADE sparsity [40], and 720p HunyuanVideo-1.5 image-to-video using its official sparse-distilled model [73]. Both workloads use $B_Q = B_{KV} = 64$.

Profiling and evaluation split. The plan tables across the four platforms use over 40k profiling cases, with no overlap with the evaluation set at the source-file and chunk level. A *chunk* indexes a mask sample within a source file. Both sets use masks from the video-model sources above.

Latency definitions. We measure latency for complete requests, kernel execution, and model workloads. A *request* is one BSA call to the runtime. *Complete-request latency* includes each backend’s preprocessing and execution through completion. *Kernel execution latency* covers the selected native CUDA implementation. Model results report the total latency for all BSA requests and a 50-step diffusion loop.

Metrics. Speedup is baseline latency divided by TESSERA latency. *Family speedup* is the ratio of mean latencies over paired cases. We report the *geometric mean* (GM) across the nine family speedups as nine-family GM and across per-case

Table 4. Kernel speedup (baseline/TESSERA latency), nine-family GM. H100 flex-block-attn covers four families; “–”: unmeasured pair.

Baseline	RTX 4090	RTX 5090	A100	H100
FlashInfer	1.15×	1.34×	1.12×	1.94×
FlexAttention	3.67×	1.70×	6.38×	2.60×
flex-block-attn	–	–	–	4.22×

speedups as paired-case GM. *Regret* is selected-plan latency divided by the fastest measured eligible-plan latency for the same case; 1× is optimal, and P95 and P99 denote the 95th and 99th percentiles. Win/tie/loss counts use a 3% tie band. TESSERA timings are medians of five requests after three untimed warmups per case, using a monotonic host timer for complete requests and CUDA events for kernel execution.

7.2 Speedups for Requests, Kernels, and Models

TESSERA *reduces complete-request latency in all 76 measured platform–baseline–family aggregates* (Figure 6). Nine-family GM complete-request speedups are 1.85–5.11× over FlashInfer and 1.62–33.73× over FlexAttention across the four GPUs. The GM speedup over flex-block-attn is 3.96× on its four supported H100 families.

TESSERA *also reduces kernel execution latency across the measured platform–baseline pairs* (Table 4). Nine-family GM kernel speedup is 1.12–1.94× over FlashInfer and 1.70–6.38× over FlexAttention; the four-family H100 GM speedup over flex-block-attn is 4.22×. Complete-request speedup exceeds kernel speedup in most cases because FlashInfer planning and FlexAttention BlockMask construction run on the request path, whereas TESSERA selects a plan from its resident plan table and prepares only that plan.

TESSERA *accelerates both attention and the full diffusion loop in the two H100 model workloads* (Figure 7). For Wan2.1 and HunyuanVideo-1.5, BSA request speedup over FlashInfer and flex-block-attn is 2.95–6.79× (panel (a)), kernel execution speedup is 2.07–3.79× (panel (b)), and 50-step diffusion-loop speedup over the same baselines is 1.22–2.08× (panel (c)). The loop gains are smaller because each step’s non-attention computation is unchanged.

7.3 Plan Performance Varies with Geometry, Mask, GPU, and Organization

Direct, Coarsened, and Refined each yield the fastest measured plan in part of the workload. Figure 8 shows the mapping type of the fastest measured plan in each platform’s native CUDA catalog for the same 2,315 masks on four GPUs.

Logical geometry changes the fastest mapping type on H100: the low-intensity Q_{16} and Q_{32} families favor Coarsened, the Q_{64} families mostly favor Direct, and the Q_{128} families favor Refined. On H100, which has the highest peak

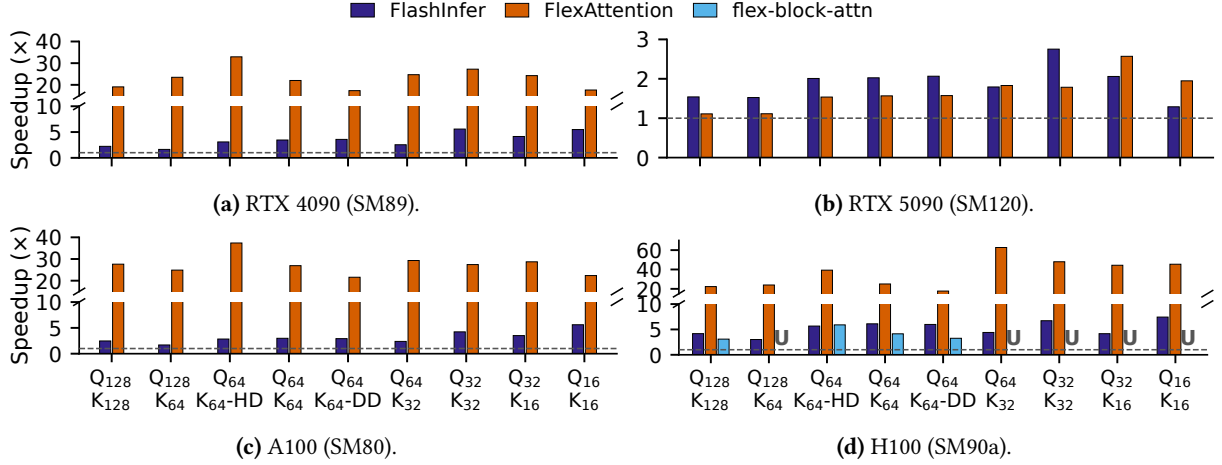


Figure 6. Complete-request speedup by workload family (baseline/TESSERA latency). Axis maxima are 40, 3, 40, and 70 $\times$ for panels (a)–(d), respectively. Panels (a), (c), and (d) omit the empty 10–15 $\times$ interval; panel (b) uses a continuous axis. Dashed lines mark 1 $\times$ speedup, and “U” marks an unsupported pair.

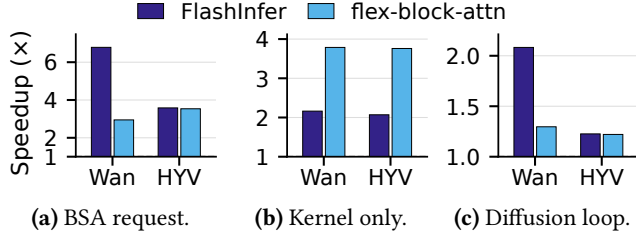


Figure 7. Model speedups on H100 (baseline/TESSERA latency) over FlashInfer and flex-block-attn for Wan2.1 (Wan) and HunyuanVideo-1.5 (HYV): (a) BSA request, (b) attention kernel execution only, and (c) the 50-step diffusion loop.

compute-to-bandwidth ratio among the evaluated GPUs (Table 2), Q128 refinement divides each logical block into smaller physical tiles near the BF16 Tensor Core instruction’s 64-row shape [50], trading additional tasks and operand traffic for a smaller per-task footprint.

Mask variation changes the fastest mapping even at fixed logical geometry and GPU. Within the H100 Q₃₂K₁₆ family, 220 masks favor Coarsened and 30 favor Direct (Figure 8).

Coarsened remains dominant across densities at fixed geometry and GPU. Across the three A100 Q₆₄K₆₄ families, Coarsened remains the fastest mapping for most cases with performance gains beyond the 3% tie band: its share is 96.8% for HD, 98.1% for the default family, and 99.6% for DD. Denser block rows amortize Direct’s data-movement and task costs over more work, changing its tradeoff with Coarsened even when the dominant mapping remains unchanged.

GPU architecture changes the fastest mapping type for the same masks. The same Q₃₂K₁₆ masks favor Coarsened on A100 and Direct on RTX 4090 for all 250 aligned masks, and RTX 5090 favors Direct for most of them (Figure 8). A100 and

Table 5. Task-organization ablation on H100 at fixed mapping (Q₁₂₈K₁₂₈, 250 masks). Kernel speedup is paired-case GM over O0; regret is relative to O2. CI denotes confidence interval; W/T/L counts wins/ties/losses against O0.

Variant	Speedup [95% CI]	Regret	W/T/L
O0: fixed base	1.000	1.1054	–
O1: TESSERA	1.105 [1.083, 1.127]	1.0003	99/151/0
O2: oracle	1.105 [1.083, 1.127]	1.0000	–

RTX 4090 favor different mappings despite nearly equal peak compute-to-bandwidth ratios (Table 2). Plan performance also depends on other aspects of the hardware configuration, such as power limits and cache capacity.

Task organization yields a 1.105 $\times$ paired-case GM kernel execution speedup over the fixed base task organization at the same mapping (Table 5). To isolate the organization layer (Section 4.3), we hold the H100 Q₁₂₈K₁₂₈ mapping to an exact 2×2 cover of physical 64×64 tiles across all 250 masks. O0 uses the fixed base task organization; O1 uses the task organization selected by TESSERA; O2 uses the *oracle*, the fastest measured eligible task organization for each case. O1 stays within 0.04% of O2 and improves 99 cases over O0 by more than the 3% tie band.

7.4 Plan Selection Preserves Gains at Low Cost

Plans selected by TESSERA have *GM regret at or below* 1.0039 on every platform (Figure 9). This corresponds to at most 0.39% higher latency in geometric-mean terms, even as the fastest mapping varies across masks and GPUs (Figure 8). Their latencies are within 3% of the minimum measured eligible-plan latency in the native CUDA catalog on 95–99%

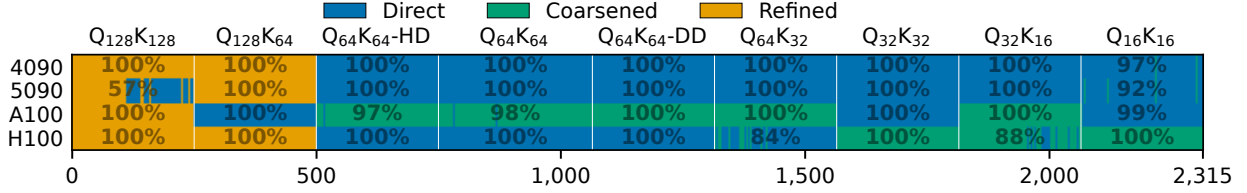


Figure 8. Mapping type of the fastest measured plan in the native CUDA catalog for 2,315 aligned masks on four GPUs. Families are ordered by Direct-tile Q-resident intensity, from 128 flop/byte for Q128 to 16 flop/byte for Q16 (Table 3). Each percentage is the share of the most frequent mapping type in that platform–family region.

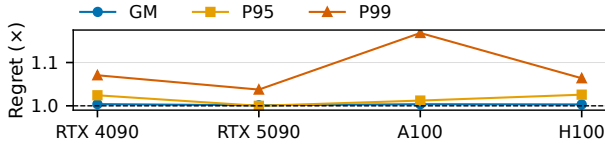


Figure 9. Selected-plan regret (selected/fastest measured eligible-plan latency): GM, P95, and P99 over 2,315 cases per platform. Lower is better; 1 is optimal.

Table 6. Planning and dispatch overhead per request on H100 for Q₆₄K₆₄.

Method	Mean (μ s)	P95 (μ s)
TESSERA dispatch	19.25	19.94
FlashInfer plan	12,160.27	37,038.77

of the 2,315 cases per platform; P99 regret ranges from 1.04 to 1.17.

We compare two preprocessing steps before kernel execution: TESSERA *dispatch* selects an eligible plan, while FlashInfer *plan()* prepares scheduling metadata. TESSERA dispatch takes 19.9 μ s at P95, while FlashInfer *plan()* takes 12.2 ms on average for the same H100 Q₆₄K₆₄ requests (Table 6). For these respective request-path preprocessing steps, TESSERA dispatch has *more than two orders of magnitude* lower mean and P95 latency.

The A0/A1/A2 ablation measures how selection quality and cost combine in complete requests on the same 2,315 H100 cases (Figure 10 and Table 7). A0 uses Direct mapping with physical tile geometry $T = B$. A1 uses *exhaustive online search*, preparing and probing every eligible native CUDA catalog candidate before executing the fastest. A2 uses TESSERA with the same native CUDA catalog and the fixed H100 plan table, preparing only the selected plan. *Plans selected by A2 closely track those selected by A1 in kernel execution speedup over A0 across the nine families.* A *cold unique request* is the first occurrence of a request, including plan selection, launch preparation, any candidate probes, and execution. *Warm exact repeats* are passes 2–4 of the same request. Four-pass amortized latency averages the cold pass and three repeats.

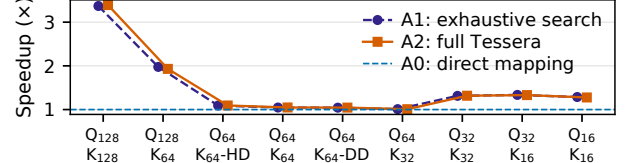


Figure 10. Kernel execution speedup over Direct (A0) for A1 and TESSERA (A2), by workload family on H100. Speedup is A0 latency divided by variant latency.

Table 7. Complete-request speedup between plan-selection variants on H100. Warm repeats cover passes 2–4; amortized covers all four passes.

Population	T_{A0}/T_{A1}	T_{A1}/T_{A2}	T_{A0}/T_{A2}
Cold unique	0.080×	14.67×	1.172×
Warm repeats	1.288×	1.212×	1.562×
Amortized	0.267×	5.374×	1.434×

TESSERA serves *cold unique requests* 14.67 $\times$ faster than A1 and 1.172 $\times$ faster than Direct (Table 7). A1 achieves only 0.080 $\times$ complete-request speedup over Direct because A1 prepares and probes every eligible plan for each cold request. The runtime reuses rankings stored in the plan table across different masks assigned to the same performance regime, selecting and preparing one plan without online probes (Section 5.2).

TESSERA serves warm exact repeats 1.212 $\times$ faster than A1 by reusing prepared plan state (Table 7). Neither method probes candidates on these repeats, but A1 still prepares its cached winner while A2 reuses the prepared plan state. Including the first cold request, TESSERA is 5.374 $\times$ faster than A1 over four passes.

8 Related Work

To our knowledge, TESSERA is the first specialized runtime system for kernel-level physical planning of dynamic block-sparse attention in video generation.

Sparse attention for video diffusion. Video sparsifiers select active interactions using spatial–temporal patterns,

semantic similarity, or learned sparsity [47, 64, 73, 76, 82, 89]. SPADE selects input-adaptive blocking and sparsity schemes and provides optimized kernels [40]. TESSERA takes their logical masks and block geometries as input.

Attention execution. Block Sparse Attention and flex-block-attn provide kernels for preset block sizes [23, 53]; FlashInfer supports multiple tile configurations and runtime planning and scheduling [83], and FlexAttention compiles programmable mask variants with configurable tiles that match or subdivide logical blocks in its standard Triton forward path [15]. TESSERA combines mappings that retain, combine, or subdivide logical blocks with task organization, and selects eligible precompiled plans per request using offline rankings. PSA decouples logical blocks from execution tiles through splitting and merging for multi-level pooled KV representations [34]. db-SP adapts multi-GPU partitioning, while FVAttn balances workloads through runtime head migration [8, 39].

9 Conclusion

TESSERA supports efficient BSA computation through flexible physical mapping and task organization that adapt execution to the mask and GPU, combined with profile-guided regime selection that chooses plans with low runtime overhead. Evaluated across four GPU generations and 2,315 real attention masks, TESSERA achieves attention speedups of up to 6.79 $\times$ over baseline systems in industrial video models.

References

- [1] Yuanhao Ban, Ruochen Wang, Tianyi Zhou, Minhao Cheng, Boqing Gong, and Cho-Jui Hsieh. 2024. Understanding the Impact of Negative Prompts: When and How Do They Take Effect?. In *European conference on computer vision*. Springer, 190–206.
- [2] Iz Beltagy, Matthew E Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150* (2020).
- [3] Daniel Bolya and Judy Hoffman. 2023. Token Merging for Fast Stable Diffusion. *arXiv:2303.17604* [cs]. doi:10.48550/arXiv.2303.17604
- [4] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Baobao Chang, Junjie Hu, and Wen Xiao. 2024. PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funnelling. *arXiv:2406.02069* [cs] <http://arxiv.org/abs/2406.02069>
- [5] Feiyang Chen, Yu Cheng, Lei Wang, Yuqing Xia, Ziming Miao, Lingxiao Ma, Fan Yang, Jilong Xue, Zhi Yang, Mao Yang, and Haibo Chen. 2025. AttentionEngine: A Versatile Framework for Efficient Attention Mechanisms on Diverse Hardware Platforms. *arXiv:2502.15349* [cs] doi:10.48550/arXiv.2502.15349
- [6] Pengtao Chen, Xianfang Zeng, Maosen Zhao, Peng Ye, Mingzhu Shen, Wei Cheng, Gang Yu, and Tao Chen. 2025. Sparse-vDiT: Unleashing the Power of Sparse Attention to Accelerate Video Diffusion Transformers. *arXiv:2506.03065* [cs]. doi:10.48550/arXiv.2506.03065
- [7] Renze Chen, Zhuofeng Wang, Beiquan Cao, Tong Wu, Size Zheng, Xiuhong Li, Xuechao Wei, Shengen Yan, Meng Li, and Yun Liang. 2024. Arkvale: Efficient generative llm inference with recallable key-value eviction. *Advances in Neural Information Processing Systems* 37 (2024), 113134–113155.
- [8] Siqi Chen, Ke Hong, Tianchen Zhao, Ruiqi Xie, Zhenhua Zhu, Xudong Zhang, and Yu Wang. 2025. db-SP: Accelerating Sparse Attention for Visual Generative Models with Dual-Balanced Sequence Parallelism. *arXiv:2511.23113* [cs.CV] <https://arxiv.org/abs/2511.23113>
- [9] Zhuoming Chen, Ranajoy Sadhukhan, Zihao Ye, Yang Zhou, Jianyu Zhang, Niklas Nolte, Yuandong Tian, Matthijs Douze, Leon Bottou, Zhihao Jia, and Beidi Chen. 2024. MagicPIG: LSH Sampling for Efficient LLM Generation. *arXiv:2410.16179* [cs.CL] <https://arxiv.org/abs/2410.16179>
- [10] Kevin Clark. 2019. What Does Bert Look At? An Analysis of Bert’s Attention. *arXiv preprint arXiv:1906.04341* (2019).
- [11] Barry Collins. 2025. “Our GPUs Are Melting,” Warns Sora Boss As New Limits Are Imposed. *Forbes*. <https://www.forbes.com/sites/barrycollins/2025/11/28/our-gpus-are-melting-warns-sora-boss-as-new-limits-are-imposed/>
- [12] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations*.
- [13] Yichuan Deng, Zhao Song, and Chiwan Yang. 2024. Attention is Naturally Sparse with Gaussian Distributed Input. *arXiv:2404.02690* [cs] <http://arxiv.org/abs/2404.02690>
- [14] Hangliang Ding, Dacheng Li, Runlong Su, Peiyuan Zhang, Zhijie Deng, Ion Stoica, and Hao Zhang. 2025. Efficient-vDiT: Efficient Video Diffusion Transformers With Attention Tile. *arXiv:2502.06155* [cs]. doi:10.48550/arXiv.2502.06155
- [15] Juechu Dong, Boyuan Feng, Driss Guessous, Yanbo Liang, and Horace He. 2024. Flex Attention: A Programming Model for Generating Optimized Attention Kernels. *arXiv:2412.05496* [cs.LG]
- [16] Jiarui Fang, Jinzhe Pan, Xibo Sun, Aoyu Li, and Jiannan Wang. 2024. xDiT: an Inference Engine for Diffusion Transformers (DiTs) with Massive Parallelism. *arXiv:2411.01738* [cs] doi:10.48550/arXiv.2411.01738
- [17] Jiarui Fang, Jinzhe Pan, Jiannan Wang, Aoyu Li, and Xibo Sun. 2024. PipeFusion: Patch-level Pipeline Parallelism for Diffusion Transformers Inference. *arXiv:2405.14430* [cs]. doi:10.48550/arXiv.2405.14430
- [18] Jiarui Fang and Shangchun Zhao. 2024. USP: A Unified Sequence Parallelism Approach for Long Context Generative AI. *arXiv:2405.07719* [cs]. doi:10.48550/arXiv.2405.07719
- [19] Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, and S. Kevin Zhou. 2024. Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation for Efficient LLM Inference. *arXiv:2407.11550* [cs] <http://arxiv.org/abs/2407.11550>
- [20] Yizhao Gao, Zhichen Zeng, Dayou Du, Shijie Cao, Peiyuan Zhou, Jiaxing Qi, Junjie Lai, Hayden Kwok-Hay So, Ting Cao, Fan Yang, and Mao Yang. 2025. SeerAttention: Learning Intrinsic Sparse Attention in Your LLMs. *arXiv:2410.13276* [cs] doi:10.48550/arXiv.2410.13276
- [21] Google. 2026. Build with Veo 3.1 Lite, Our Most Cost-Effective Video Generation Model. *Google Blog*. <https://blog.google/innovation-and-ai/technology/ai/veo-3-1-lite/>
- [22] Google DeepMind. 2026. Veo 3.1. *Google DeepMind Models*. <https://deepmind.google/models/veo/>
- [23] Junxian Guo, Haotian Tang, Shang Yang, Zhekai Zhang, Zhijian Liu, and Song Han. 2024. Block Sparse Attention. <https://github.com/mit-han-lab/Block-Sparse-Attention>.
- [24] Tae Jun Ham, Juan L. Aragón, and Margaret Martonosi. 2015. DeSC: Decoupled Supply-Compute Communication Management for Heterogeneous Architectures. In *Proceedings of the 48th International Symposium on Microarchitecture (MICRO)*. 191–203. doi:10.1145/2830772.2830800
- [25] Ziqi Huang, Yanan He, Jiahuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chanpaisit, et al. 2024. Vbench: Comprehensive benchmark suite for video generative models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 21807–21818.

- [26] Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H. Abdi, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. MInference 1.0: Accelerating Pre-filling for Long-Context LLMs via Dynamic Sparse Attention. arXiv:2407.02490 [cs] <http://arxiv.org/abs/2407.02490>
- [27] Minchul Kim, Shangqian Gao, Yen-Chang Hsu, Yilin Shen, and Hongxia Jin. 2024. Token Fusion: Bridging the Gap between Token Pruning and Token Merging. In *2024 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. IEEE, Waikoloa, HI, USA, 1372–1381. doi:10.1109/WACV57701.2024.00141
- [28] Diederik P. Kingma and Max Welling. 2014. Auto-Encoding Variational Bayes. In *International Conference on Learning Representations (ICLR)*.
- [29] Weijie Kong, Qi Tian, Zijian Zhang, Rox Min, Zuozhuo Dai, Jin Zhou, Jiangfeng Xiong, Xin Li, Bo Wu, Jianwei Zhang, Kathrina Wu, Qin Lin, Junkun Yuan, Yanxin Long, Aladdin Wang, Andong Wang, Changlin Li, Duojuan Huang, Fang Yang, Hao Tan, Hongmei Wang, Jacob Song, Jiawang Bai, Jianbing Wu, Jinbao Xue, Joey Wang, Kai Wang, Mengyang Liu, Pengyu Li, Shuai Li, Weiyan Wang, Wenqing Yu, Xinchu Deng, Yang Li, Yi Chen, Yutao Cui, Yuanbo Peng, Zhentao Yu, Zhiyu He, Zhiyong Xu, Zixiang Zhou, Zunnan Xu, Yangyu Tao, Qinglin Lu, Songtao Liu, Dax Zhou, Hongfa Wang, Yong Yang, Di Wang, Yuhong Liu, Jie Jiang, and Caesar Zhong. 2024. HunyuanVideo: A Systematic Framework for Large Video Generative Models. arXiv:2412.03603 [cs.CV]
- [30] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. arXiv:2309.06180 [cs] <http://arxiv.org/abs/2309.06180>
- [31] Xunhao Lai, Jianqiao Lu, Yao Luo, Yiyuan Ma, and Xun Zhou. 2025. FlexPrefill: A Context-Aware Sparse Attention Mechanism for Efficient Long-Sequence Inference. arXiv:2502.20766 [cs] doi:10.48550/arXiv.2502.20766
- [32] Dongyeun Lee, Amir Zandieh, Vahab Mirrokni, Junmo Kim, and Insu Han. 2026. HyperVAttention: Efficient Sparse Attention with Spatio-Temporal Clustering for Video Diffusion. arXiv preprint arXiv:2607.03012 (2026).
- [33] Muiyang Li, Tianle Cai, Jiaxin Cao, Qingsheng Zhang, Han Cai, Junjie Bai, Yangqing Jia, Kai Li, and Song Han. 2024. DistriFusion: Distributed parallel inference for high-resolution diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 7183–7193.
- [34] Xiaolong Li, Youping Gu, Xi Lin, Weijie Wang, and Bohan Zhuang. 2025. PSA: Pyramid Sparse Attention for Efficient Video Understanding and Generation. arXiv:2512.04025 [cs.CV] <https://arxiv.org/abs/2512.04025>
- [35] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. SnapKV: LLM Knows What You are Looking for Before Generation. arXiv:2404.14469 [cs] <http://arxiv.org/abs/2404.14469>
- [36] Zhiyao Li, Jiaxiang Li, Taijie Chen, Dimin Niu, Hongzhong Zheng, Yuan Xie, and Mingyu Gao. 2023. Spada: Accelerating Sparse Matrix Multiplication with Adaptive Dataflow. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 747–761. doi:10.1145/3575693.3575706
- [37] Anmin Liu, Ruixuan Yang, Huiqiang Jiang, Bin Lin, Minmin Sun, Yong Li, Chen Zhang, and Tao Xie. 2026. VecAttention: Vector-wise Sparse Attention for Accelerating Long Context Inference. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [38] Fangfu Liu, Hanyang Wang, Yimo Cai, Kaiyan Zhang, Xiaohang Zhan, and Yueqi Duan. 2025. Video-T1: Test-Time Scaling for Video Generation. arXiv:2503.18942 [cs]. doi:10.48550/arXiv.2503.18942
- [39] Hao Liu, Chenghuan Huang, Ye Huang, Zhiying Wen, Hao Liu, Mohan Zhang, Chen Li, Ziyang Ma, Jing Lyu, and Jiangsu Du. 2026. FVAttn: Adaptive Sparse Attention with Runtime Load Balancing for Video Generation. arXiv:2607.16190 [cs.CV] <https://arxiv.org/abs/2607.16190>
- [40] Shanghao Liu, Renze Chen, Size Zheng, Yuanqiang Liu, Yun (Eric) Liang, and Hailong Yang. 2026. SPADE: An Input-Adaptive Sparse Attention Engine for Fast Video Diffusion Models Inference. In *Proceedings of the 63rd ACM/IEEE Design Automation Conference*. doi:10.1145/3770743.3804015
- [41] Zichang Liu, Jue Wang, Tri Dao, Tianyi Zhou, Binhang Yuan, Zhao Song, Anshumali Shrivastava, Ce Zhang, Yuandong Tian, Christopher Re, et al. 2023. Deja vu: Contextual sparsity for efficient llms at inference time. In *International Conference on Machine Learning*. PMLR, 22137–22176.
- [42] Junlin Lv, Yuan Feng, Xike Xie, Xin Jia, Qirong Peng, and Guiming Xie. 2024. CritiPrefill: A Segment-wise Criticality-based Approach for Prefilling Acceleration in LLMs. arXiv:2409.12490 [cs] <http://arxiv.org/abs/2409.12490>
- [43] Zhengyao Lv, Chenyang Si, Junhao Song, Zhenyu Yang, Yu Qiao, Ziwei Liu, and Kwan-Yee K Wong. 2024. Fastercache: Training-free video diffusion model acceleration with high quality. arXiv preprint arXiv:2410.19355 (2024).
- [44] Xinyin Ma, Gongfan Fang, Michael Bi Mi, and Xinchao Wang. 2024. Learning-to-Cache: Accelerating Diffusion Transformer via Layer Caching. arXiv:2406.01733 [cs]. doi:10.48550/arXiv.2406.01733
- [45] Xinyin Ma, Gongfan Fang, and Xinchao Wang. 2024. DeepCache: Accelerating Diffusion Models for Free. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 15762–15772. https://openaccess.thecvf.com/content/CVPR2024/html/Ma_DeepCache_Accelerating_Diffusion_Models_for_Free_CVPR_2024_paper.html
- [46] Yixuan Mei, Yonghao Zhuang, Xupeng Miao, Juncheng Yang, Zhihao Jia, and Rashmi Vinayak. 2025. Helix: Serving Large Language Models over Heterogeneous GPUs and Network via Max-Flow. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. Association for Computing Machinery, New York, NY, USA, 586–602. doi:10.1145/3669940.3707215
- [47] Meituan LongCat Team, Xunliang Cai, Qilong Huang, Zhuoliang Kang, Hongyu Li, Shijun Liang, Liya Ma, Siyu Ren, Xiaoming Wei, Rixu Xie, and Tong Zhang. 2025. LongCat-Video Technical Report. arXiv:2510.22200 [cs.CV] <https://arxiv.org/abs/2510.22200>
- [48] Francisco Muñoz-Martínez, Raveesh Garg, Michael Pellauer, José L. Abellán, Manuel E. Acacio, and Tushar Krishna. 2023. Flexagon: A Multi-Dataflow Sparse-Sparse Matrix Multiplication Accelerator for Efficient DNN Processing. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 252–265. doi:10.1145/3582016.3582069
- [49] NVIDIA Corporation. 2025. Efficient GEMM in CUDA. NVIDIA CUTLASS Documentation. Accessed 2026-09-02. https://docs.nvidia.com/cutlass/media/docs/cpp/efficient_gemm.html
- [50] NVIDIA Corporation. 2025. Warpgroup MMA Programming Guide. NVIDIA CUTLASS Documentation. Accessed 2026-09-02. https://docs.nvidia.com/cutlass/media/docs/pythonDSL/mma_docs/wgmma_programming.html
- [51] Muhammad Osama, Duane Merrill, Cris Cecka, Michael Garland, and John D. Owens. 2023. Stream-K: Work-Centric Parallel Decomposition for Dense Matrix-Matrix Multiplication on the GPU. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP)*. 429–431. doi:10.1145/3572848.3577479
- [52] William Peebles and Saining Xie. 2023. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*. 4195–4205.

- [53] Yuanbo Peng, Penghao Zhao, Jiangfeng Xiong, Songtao Liu, Fang Yang, Jianbing Wu, Zhao Zhong, Linus Key, Peng Chen, and Jie Jiang. 2025. flex-block-attn: An Efficient Block-Sparse Attention Communication Library. Software artifact. <https://github.com/Tencent-Hunyuan/flex-block-attn>
- [54] Seedance Team. 2026. Seedance 2.0: Advancing Video Generation for World Complexity. arXiv:2604.14148 [cs.CV] <https://arxiv.org/abs/2604.14148>
- [55] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2025. FlashAttention-3: fast and accurate attention with asynchrony and low-precision. In *Proceedings of the 38th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) (NIPS '24). Curran Associates Inc., Red Hook, NY, USA, Article 2193, 28 pages.
- [56] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed Chi, Nathanael Schärli, and Denny Zhou. 2023. Large Language Models Can Be Easily Distracted by Irrelevant Context. arXiv:2302.00093 [cs] <http://arxiv.org/abs/2302.00093>
- [57] Han Shi, Jiahui Gao, Xiaozhe Ren, Hang Xu, Xiaodan Liang, Zhen-guo Li, and James Tin-Yau Kwok. 2021. SparseBERT: Rethinking the Importance Analysis in Self-attention. In *Proceedings of the 38th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 139)*, Marina Meila and Tong Zhang (Eds.). PMLR, 9547–9557. <https://proceedings.mlr.press/v139/shi21a.html>
- [58] James E. Smith. 1982. Decoupled Access/Execute Computer Architectures. In *Proceedings of the 9th Annual Symposium on Computer Architecture (ISCA)*. 112–119.
- [59] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. 2023. PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU. arXiv:2312.12456 [cs.LG]
- [60] Benjamin Spector, Simran Arora, Aaryan Singhal, Arjun Parthasarathy, Dan Fu, and Christopher Re. 2025. ThunderKittens: Simple, Fast, and {Adorable} Kernels. In *International Conference on Learning Representations*, Vol. 2025. 2097–2130.
- [61] Jovan Stojkovic, Chaojie Zhang, Iñigo Goiri, Esha Choukse, Haoran Qiu, Rodrigo Fonseca, Josep Torrellas, and Ricardo Bianchini. 2025. TAPAS: Thermal- and Power-Aware Scheduling for LLM Inference in Cloud Platforms. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. Association for Computing Machinery, New York, NY, USA, 1266–1281. doi:10.1145/3676641.3716025
- [62] Xibo Sun, Jiarui Fang, Aoyu Li, and Jinzhe Pan. 2024. Unveiling Redundancy in Diffusion Transformers (DiTs): A Systematic Study. arXiv:2411.13588 [cs] doi:10.48550/arXiv.2411.13588
- [63] Haoyue Tan, Shengnan Wang, Yulin Qiao, Juncheng Zhang, Youhui Bai, Ping Gong, Zewen Jin, and Cheng Li. 2026. AdaCluster: Adaptive Query-Key Clustering for Sparse Attention in Video Generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 43249–43259.
- [64] Xin Tan, Yuetao Chen, Yimin Jiang, Xing Chen, Kun Yan, Nan Duan, Yibo Zhu, Daxin Jiang, and Hong Xu. 2026. Dynamic Sparsity in Large-Scale Video DiT Training. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. Association for Computing Machinery, New York, NY, USA, 101–116. doi:10.1145/3760250.3762216
- [65] Hansi Teng, Hongyu Jia, Lei Sun, Lingzhi Li, Maolin Li, Mingqiu Tang, Shuai Han, Tianning Zhang, WQ Zhang, Weifeng Luo, et al. 2025. MAGI-1: Autoregressive Video Generation at Scale. arXiv preprint arXiv:2505.13211 (2025).
- [66] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*. 10–19. doi:10.1145/3315508.3329973
- [67] Dezhao Tu, Danylo Vashchilenko, Yuzhe Lu, and Panpan Xu. 2024. VL-Cache: Sparsity and Modality-Aware KV Cache Compression for Vision-Language Model Inference Acceleration. arXiv preprint arXiv:2410.23317 (2024).
- [68] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems*.
- [69] Wan Team, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Feiwu Yu, Haiming Zhao, Jianxiao Yang, Jianyuan Zeng, Jiayu Wang, Jingfeng Zhang, Jingren Zhou, Jinkai Wang, Jixuan Chen, Kai Zhu, Kang Zhao, Keyu Yan, Lianghua Huang, Mengyang Feng, Ningyi Zhang, Pandeng Li, Pingyu Wu, Ruihang Chu, Ruili Feng, Shiwei Zhang, Siyang Sun, Tao Fang, Tianxing Wang, Tianyi Gui, Tingyu Weng, Tong Shen, Wei Lin, Wei Wang, Wei Wang, Wenmeng Zhou, Wenten Wang, Wenting Shen, Wenyuan Yu, Xianzhong Shi, Xiaoming Huang, Xin Xu, Yan Kou, Yangyu Lv, Yifei Li, Yijing Liu, Yiming Wang, Yingya Zhang, Yitong Huang, Yong Li, You Wu, Yu Liu, Yulin Pan, Yun Zheng, Yuntao Hong, Yupeng Shi, Yutong Feng, Zeyinzi Jiang, Zhen Han, Zhi-Fan Wu, and Ziyu Liu. 2025. Wan: Open and Advanced Large-Scale Video Generative Models. arXiv:2503.20314 [cs.CV]
- [70] Guoxia Wang, Jinle Zeng, Xiyuan Xiao, Siming Wu, Jiabin Yang, Lujing Zheng, Zeyu Chen, Jiang Bian, Dianhai Yu, and Haifeng Wang. 2025. FlashMask: Efficient and Rich Mask Extension of FlashAttention. In *International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2025/hash/44c624d8f4e802453ad12593b2aa8e38-Abstract-Conference.html
- [71] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Commun. ACM* 52, 4 (2009), 65–76. doi:10.1145/1498765.1498785
- [72] Jaeyeon Won, Willow Ahrens, Joel S. Emer, and Saman Amarasinghe. 2026. Insum: Sparse GPU Kernels Simplified and Optimized with Indirect Einsums. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. Association for Computing Machinery, New York, NY, USA, 993–1006. doi:10.1145/3779212.3790176
- [73] Bing Wu, Chang Zou, Changlin Li, Duojuan Huang, Fang Yang, Hao Tan, Jack Peng, Jianbing Wu, Jiangfeng Xiong, Jie Jiang, Linus, Patrol, Peizhen Zhang, Peng Chen, Penghao Zhao, Qi Tian, Songtao Liu, Weijie Kong, Weiyan Wang, Xiao He, Xin Li, Xincheng Deng, Xuefei Zhe, Yang Li, Yanxin Long, Yuanbo Peng, Yue Wu, Yuhong Liu, Zhenyu Wang, Zuozhuo Dai, Bo Peng, Coopers Li, Gu Gong, Guojian Xiao, Jiahe Tian, Jiaxin Lin, Jie Liu, Jihong Zhang, Jiesong Lian, Kaihang Pan, Lei Wang, Lin Niu, Mingtao Chen, Mingyang Chen, Mingzhe Zheng, Miles Yang, Qiangqiang Hu, Qi Yang, Qiuyong Xiao, Runzhou Wu, Ryan Xu, Rui Yuan, Shanshan Sang, Shisheng Huang, Siruis Gong, Shuo Huang, Weiting Guo, Xiang Yuan, Xiaojia Chen, Xiawei Hu, Wenzhi Sun, Xiele Wu, Xianshun Ren, Xiaoyan Yuan, Xiaoyue Mi, Yepeng Zhang, Yifu Sun, Yiting Lu, Yitong Li, You Huang, Yu Tang, Yixuan Li, Yuhang Deng, Yuan Zhou, Zhichao Hu, Zhiguang Liu, Zhihe Yang, Zilin Yang, Zhenzhi Lu, Zixiang Zhou, and Zhao Zhong. 2025. HunyuanVideo 1.5 Technical Report. arXiv:2511.18870 [cs.CV]
- [74] Pingyu Wu, Kai Zhu, Yu Liu, Liming Zhao, Wei Zhai, Yang Cao, and Zheng-Jun Zha. 2025. Improved video vae for latent video diffusion model. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 18124–18133.
- [75] Yuxin Wu and Kaiming He. 2018. Group normalization. In *Proceedings of the European conference on computer vision (ECCV)*. 3–19.
- [76] Haocheng Xi, Shuo Yang, Yilong Zhao, Chenfeng Xu, Muyang Li, Xiuyu Li, Yujun Lin, Han Cai, Jintao Zhang, Dacheng Li, Jianfei Chen, Ion Stoica, Kurt Keutzer, and Song Han. 2025. Sparse Video-Gen: Accelerating Video Diffusion Transformers with Spatial-Temporal Sparsity. In *Proceedings of the 42nd International Conference on Machine*

- Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 68208–68224. <https://proceedings.mlr.press/v267/xi25c.html>
- [77] Chaojun Xiao, Pengle Zhang, Xu Han, Guangxuan Xiao, Yankai Lin, Zhengyan Zhang, Zhiyuan Liu, and Maosong Sun. 2024. InfLLM: Training-Free Long-Context Extrapolation for LLMs with an Efficient Context Memory. *arXiv:2402.04617 [cs]* <http://arxiv.org/abs/2402.04617>
- [78] Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Junxian Guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. 2024. DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads. *arXiv:2410.10819 [cs]* <http://arxiv.org/abs/2410.10819>
- [79] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient Streaming Language Models with Attention Sinks. *arXiv:2309.17453 [cs]* <http://arxiv.org/abs/2309.17453>
- [80] Ruyi Xu, Guangxuan Xiao, Haofeng Huang, Junxian Guo, and Song Han. 2025. XAttention: Block Sparse Attention with Antidiagonal Scoring. *arXiv:2503.16428 [cs]*. doi:10.48550/arXiv.2503.16428
- [81] Rohan Yadav, Michael Bauer, David Broman, Michael Garland, Alex Aiken, and Fredrik Kjolstad. 2025. Task-Based Tensor Computations on Modern GPUs. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. doi:10.1145/3729300
- [82] Shuo Yang, Haocheng Xi, Yilong Zhao, Muiyang Li, Jintao Zhang, Han Cai, Yujun Lin, Xiuyu Li, Chenfeng Xu, Kelly Peng, Jianfei Chen, Song Han, Kurt Keutzer, and Ion Stoica. 2025. Sparse VideoGen2: Accelerate Video Generation with Sparse Attention via Semantic-Aware Permutation. In *Advances in Neural Information Processing Systems*, Vol. 38. doi:10.52202/085713-3243
- [83] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. In *Proceedings of Machine Learning and Systems*.
- [84] Zihao Ye, Ruihang Lai, Junru Shao, Tianqi Chen, and Luis Ceze. 2023. SparseTIR: Composable Abstractions for Sparse Compilation in Deep Learning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. Association for Computing Machinery, New York, NY, USA, 660–678. doi:10.1145/3582016.3582047
- [85] Zhihang Yuan, Hanling Zhang, Lu Pu, Xuefei Ning, Linfeng Zhang, Tianchen Zhao, Shengen Yan, Guohao Dai, and Yu Wang. 2024. Dittfastattn: Attention compression for diffusion transformer models. *Advances in Neural Information Processing Systems* 37 (2024), 1196–1219.
- [86] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. 2020. Big Bird: Transformers for Longer Sequences. In *Advances in Neural Information Processing Systems*, Vol. 33. 17283–17297. <https://papers.neurips.cc/paper/2020/hash/c8512d142a2d849725f31a9a7a361ab9-Abstract.html>
- [87] Hanling Zhang, Rundong Su, Zhihang Yuan, Pengtao Chen, Mingzhu Shen Yibo Fan, Shengen Yan, Guohao Dai, and Yu Wang. 2025. DiT-FastAttnV2: Head-wise Attention Compression for Multi-Modality Diffusion Transformers. *arXiv preprint arXiv:2503.22796* (2025).
- [88] Jintao Zhang, Chendong Xiang, Haofeng Huang, Jia Wei, Haocheng Xi, Jun Zhu, and Jianfei Chen. 2025. SpargeAttention: Accurate and Training-free Sparse Attention Accelerating Any Model Inference. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 76397–76413. <https://proceedings.mlr.press/v267/zhang25ch.html>
- [89] Peiyuan Zhang, Yongqi Chen, Haofeng Huang, Will Lin, Zhengzhong Liu, Ion Stoica, Eric P. Xing, and Hao Helen Zhang. 2025. Faster Video Diffusion with Trainable Sparse Attention. doi:10.52202/085713-5099
- [90] Peiyuan Zhang, Yongqi Chen, Runlong Su, Hangliang Ding, Ion Stoica, Zhenghong Liu, and Hao Zhang. 2025. Fast Video Generation with Sliding Tile Attention. *arXiv:2502.04507 [cs.CV]*
- [91] Dian Zheng, Ziqi Huang, Hongbo Liu, Kai Zou, Yinan He, Fan Zhang, Lulu Gu, Yuanhan Zhang, Jingwen He, Wei-Shi Zheng, et al. 2025. Vbench-2.0: Advancing video generation benchmark suite for intrinsic faithfulness. *arXiv preprint arXiv:2503.21755* (2025).
- [92] Keren Zhou, Mario Lezcano-Casado, Adam P. Goucher, Akhmed Rakhmati, Jeff Niu, Justin Lebar, Pawel Szczerebuk, Peter Bell, Phil Tillet, Thomas Raoux, and Zahi Moudallal. 2026. Linear Layouts: Robust Code Generation of Efficient Tensor Computation Using $\mathbb{F}_2$. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. Association for Computing Machinery, New York, NY, USA, 132–146. doi:10.1145/3760250.3762221
- [93] Zihan Zou, Xinming Yan, Shun Zhang, Peng Zheng, Guang Yang, Hao Cai, and Bo Liu. 2025. S-DMA: Sparse Diffusion Models Acceleration via Spatiality-Aware Prediction and Dimension-Adaptive Dataflow. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25)*. Association for Computing Machinery, New York, NY, USA, 432–444. doi:10.1145/3725843.3756046